

Name :- Rucha Nitin Pawar

Branch :- AI&DS

Div :- B

Roll No :- 70

Practical Batch :- B4

Aim:- Data Visualization with Matplotlib

Theory

1. What is the importance of data visualization in data analysis? How does visualization help in exploratory data analysis (EDA)?

Data visualization is the process of representing data in graphical form such as charts, graphs, and plots. It is important because it makes complex data easy to understand and interpret. Visualization helps in identifying patterns, trends, and relationships within the data, which may not be visible in raw data form. In exploratory data analysis (EDA), visualization plays a key role by helping to detect outliers, understand data distribution, and analyze relationships between variables. It also helps in identifying errors or missing values in the dataset, making the analysis more effective.

2. What is the difference between Matplotlib and other visualization libraries?

Matplotlib is a low-level data visualization library in Python that provides high flexibility and control over plotting. Compared to other libraries like Seaborn and Plotly, Matplotlib requires more code but allows detailed customization. Other libraries are higher-level and easier to use, offering more attractive and modern visualizations with less effort. Seaborn is built on top of Matplotlib and provides better default styles, while Plotly offers interactive visualizations. Thus, Matplotlib is preferred when detailed customization is needed, whereas other libraries are used for quick and visually appealing plots.

3. What is the purpose of the pyplot module in Matplotlib?

The pyplot module in Matplotlib is used to create and manage plots easily. It provides a collection of functions that allow users to generate different types of graphs such as line plots, bar charts, and scatter plots. It also helps in adding titles, labels, legends, and grids to the graphs. The pyplot module works similarly to MATLAB, making it simple to use for beginners. The function `plt.show()` is used to display the final plot on the screen.

4. Explain the process of creating a basic line plot?

The process of creating a basic line plot involves several steps. First, the `matplotlib.pyplot` module is imported. Then, the data for the x-axis and y-axis is defined. After that, the `plot()` function is used to draw the line graph. Labels for the x-axis and y-axis are added using `xlabel()` and `ylabel()` functions, and a title is given using the `title()` function. Finally, the `show()` function is used to display the graph. Line plots are mainly used to show trends or changes over time.

5. Why is customization important in data visualization? What customization options are available in Matplotlib?

Customization is important in data visualization because it improves the clarity, readability, and overall appearance of the graphs. It helps in highlighting important information and makes the visualization more understandable for the audience. Matplotlib provides various customization options such as adding titles, labeling axes, changing colors, modifying line styles, adding markers, including legends, and displaying grids. These features make the graphs more informative and visually appealing, which enhances the effectiveness of data presentation.

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

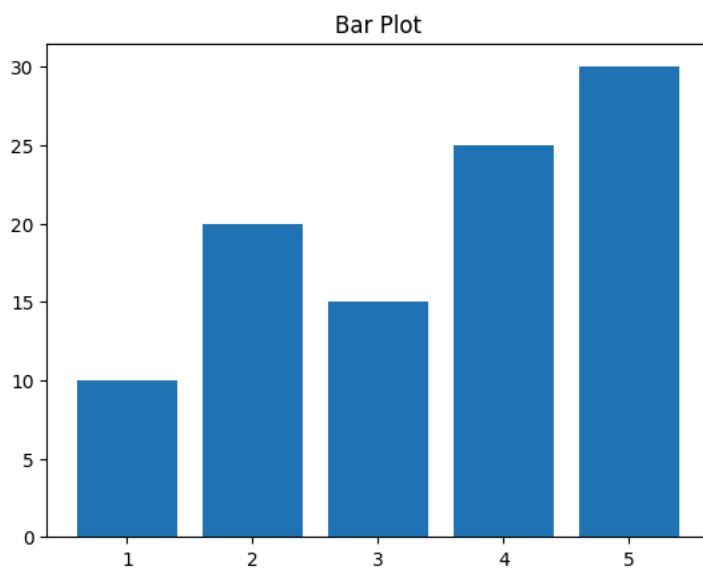
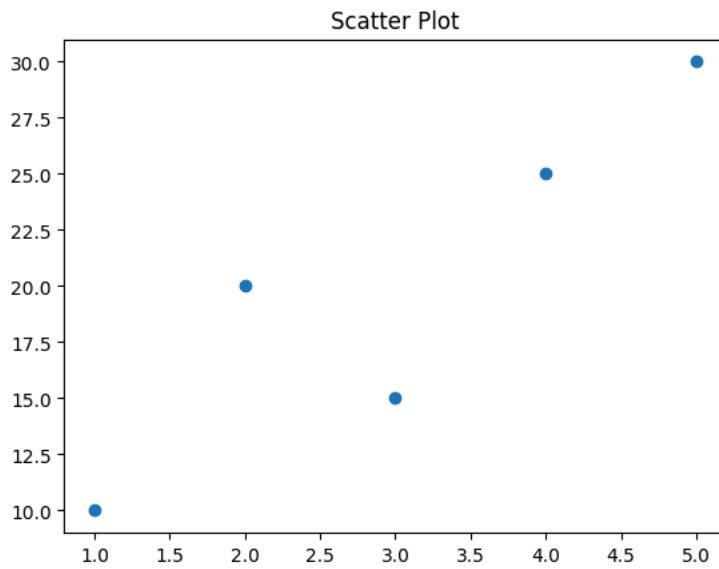
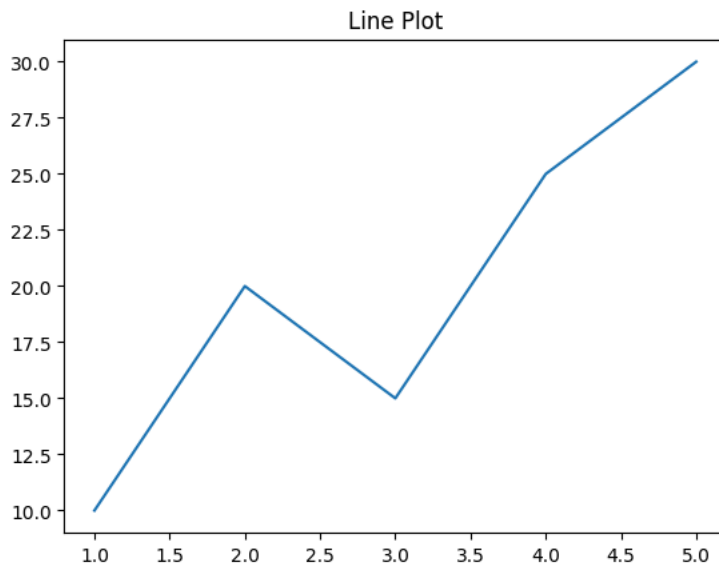
x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 25, 30]

plt.figure()
plt.plot(x, y)
plt.title("Line Plot")
plt.show()

plt.figure()
plt.scatter(x, y)
plt.title("Scatter Plot")
plt.show()

plt.figure()
plt.bar(x, y)
```

```
plt.title("Bar Plot")
plt.show()
```

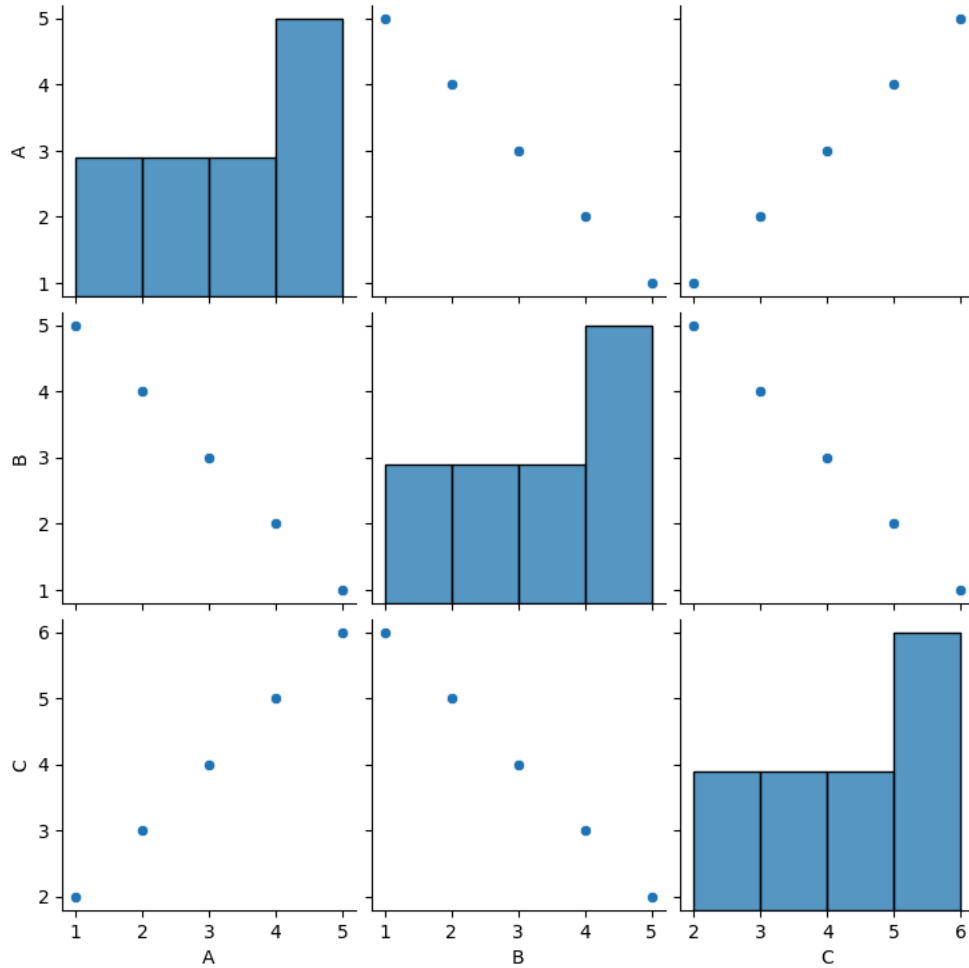


```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
import pandas as pd

df = pd.DataFrame({
    'A': [1, 2, 3, 4, 5],
    'B': [5, 4, 3, 2, 1],
    'C': [2, 3, 4, 5, 6]
})

sns.pairplot(df)
plt.show()
```

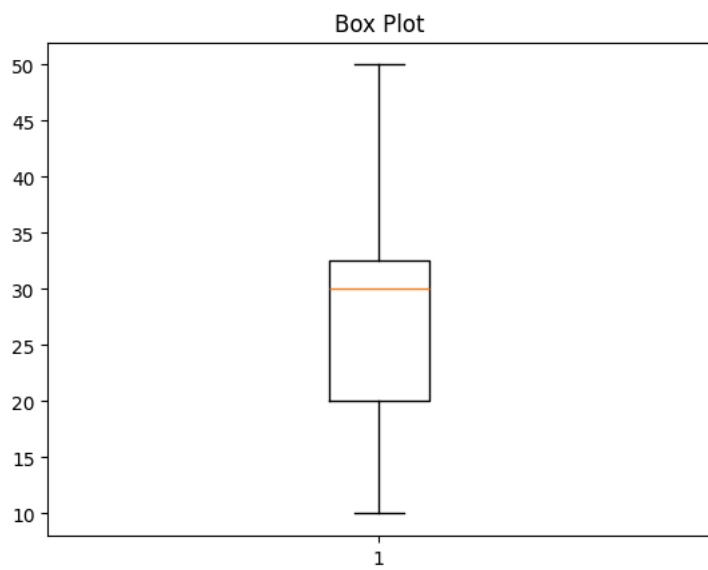
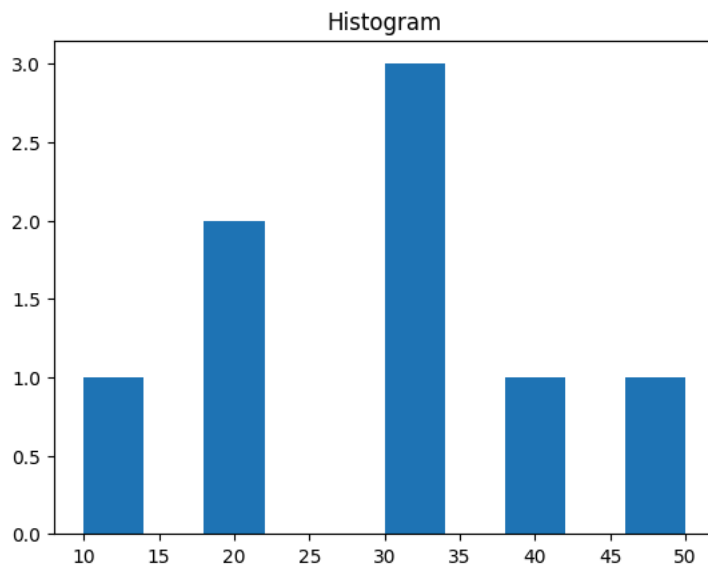


```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

data = [10, 20, 20, 30, 30, 30, 40, 50]

plt.figure()
plt.hist(data)
plt.title("Histogram")
plt.show()

plt.figure()
plt.boxplot(data)
plt.title("Box Plot")
plt.show()
```



Conclusion:

This practical demonstrated how to use Matplotlib for data visualization in Python. It helped in creating different types of plots such as line, scatter, bar, histogram, and box plot. These visualizations make data easier to understand by showing patterns, trends, and comparisons clearly, which is useful for analysis and decision-making.